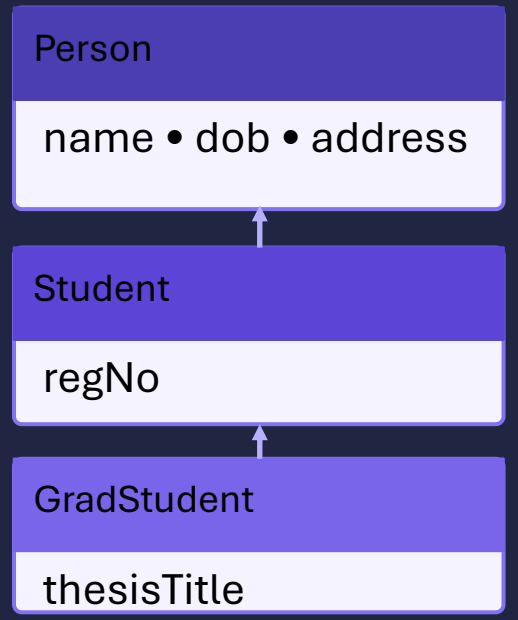
CSC241 • OBJECT-ORIENTED PROGRAMMING

Lecture 13

Inheritance in Java — Comprehensive

extends • super • overriding • subtype behavior • String • wrappers



BCS • Fall 2026

Dr Muhammad Shahid Bhatti
Assistant Professor • Department of Computer Science
COMSATS University Islamabad, Lahore Campus

Built around the Person → Student → GradStudent code used in class

Bridge from the previous class

You already wrote a working hierarchy. Today we explain what Java is doing underneath it.

What students already did

- Person as the superclass
- Student extends Person
- GradStudent extends Student
- Used super(...) in constructors
- Instantiated all three types in Demo

What was still “black box”

- What is really inside a GradStudent object?
- Why must Person be constructed first?
- What happens if Person has no no-arg constructor?
- What does private mean inside a hierarchy?
- Is reuse alone a valid reason for inheritance?

Today’s deeper goal

Move from “I can write extends” to “I can reason about a hierarchy, trace construction, diagnose errors, and defend the design.”

LECTURE 13

Lecture 13 scope — aligned with the current lecture-wise plan

Extended edition: inheritance, overriding, and related design concepts; no interfaces.

- Single, multilevel, and hierarchical inheritance
- extends and super — syntax plus meaning
- Constructor chaining in inheritance hierarchies
- Reuse versus subtype relationship — not the same design goal
- Superclass private state and access through superclass behavior
- UML generalization ↔ Java extends mapping
- Fragile-base-class intuition and the cost of deep hierarchies

Extended continuation
Overriding, @Override, protected, final and polymorphism foundations follow after the original inheritance foundation.

Teaching order
First establish inherited state and construction; then study overriding and runtime behavior, followed by String and wrappers.

Reading anchor: Liang, Chapter 11

Learning outcomes

By the end, students should be able to explain both code and design decisions.

TRACE

Trace Person → Student → GradStudent construction and explain why constructor bodies complete in superclass-to-subclass order.

DIAGNOSE

Predict common compiler errors involving implicit super(), unavailable constructors, and direct access to private superclass fields.

MODEL

Read a UML generalization tree and identify which state/behavior is introduced at each level.

DESIGN

Distinguish IS-A from HAS-A and explain why code reuse alone does not justify inheritance.

EVALUATE

Recognize hierarchies that are too deep, semantically weak, or risky to change.

PART I

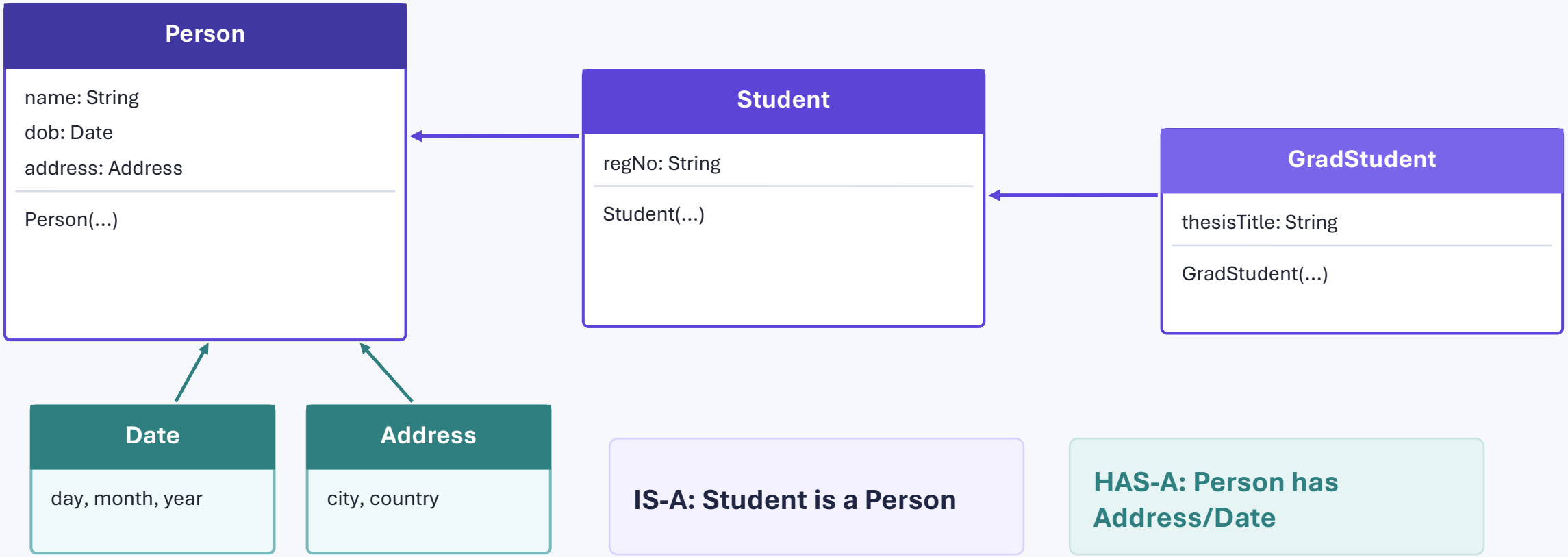
Read the hierarchy, not just the syntax

Start from the exact model already used in class: Person → Student → GradStudent, with Address and Date as contained objects.

Predict → Trace → Compile → Explain

Your current class model

One example already contains two different object relationships: IS-A and HAS-A.



Read every inheritance arrow as an English sentence

The sentence must make sense before the Java syntax does.

Student IS-A Person

GradStudent IS-A Student

Therefore GradStudent IS-A Person

Now contrast with containment:

Person HAS-A Address

Person HAS-A Date of birth

Design habit: say the relationship aloud before choosing extends.

What is inside one GradStudent object?

Think of one object with state introduced at different levels of the hierarchy.

GradStudent object gstd1

Person portion	name dob address
Student portion	regNo
GradStudent portion	thesisTitle

One object — not three separate objects

Each superclass contributes state/behavior to the final subtype object.

Construction still occurs level by level.

“Inherited” has a precise meaning

Do not confuse object state, inherited members, and constructor execution.

Inherited accessible members

A subclass can use accessible superclass methods/fields according to Java access rules.

Example: a public getName() method declared in Person can be called using a Student object.

Private superclass state

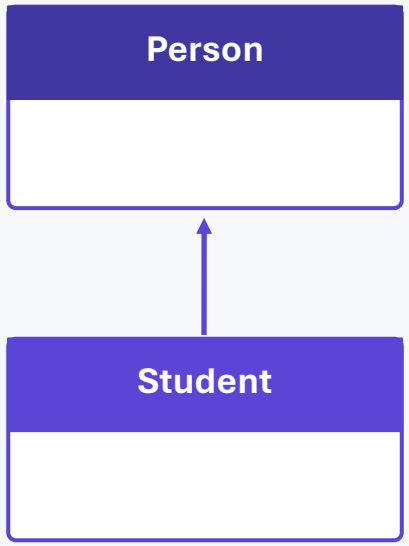
Private fields are part of the superclass portion of a subclass object, but they are not inherited as members directly accessible in subclass source code.

Constructors

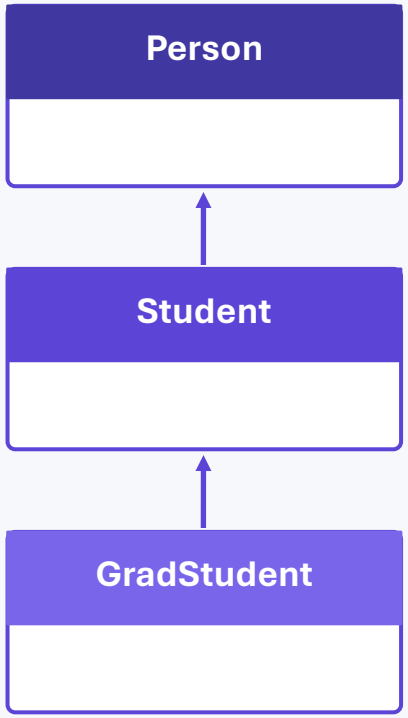
Constructors are not inherited. A subclass constructor selects or implicitly invokes a constructor of its direct superclass.

Three class-inheritance shapes used in Java

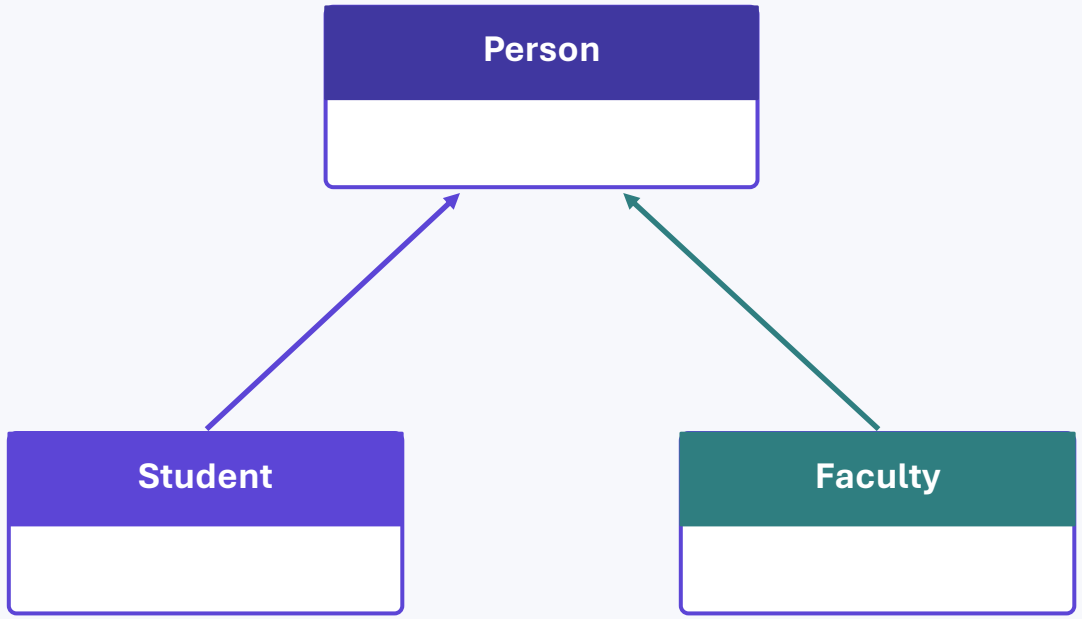
All three are built from the same rule: each class has at most one direct superclass.



Single



Multilevel

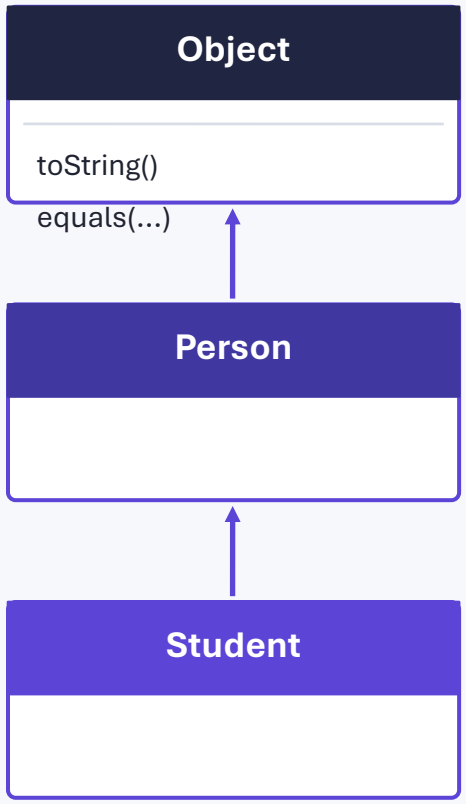


Hierarchical

Java class inheritance is single-direct-parent, not multiple-class inheritance.

There is always a superclass chain

If you do not explicitly extend another class, the class ultimately derives from java.lang.Object.



Hierarchy syntax

```
1 public class Person {
2     // implicitly: extends
Object
3 }
4
5 public class Student extends
Person {
6     // direct superclass is
Person
7 }
```

Direct superclass
Student → Person

Indirect superclass
Student → Object

Important: one direct superclass does not mean only one level in the total inheritance chain.

PART II

Constructor chaining — the heart of today's deep dive

Creating a subclass object means constructing the inherited superclass portion correctly before the subclass finishes construction.

Predict → Trace → Compile → Explain

Your current constructors already form a chain

Follow only the super(...) calls — they tell us which superclass constructor is selected.

Person.java

```
1 Person(String name, Date dob,
Address address) {
2     this.name = name;
3     this.address = address;
4     this.dob = dob;
5 }
```

Student.java

```
1 Student(String name, Date dob,
2         Address address, String
regNo) {
3     super(name, dob, address);
4     this.regNo = regNo;
5 }
```

GradStudent.java

```
1 GradStudent(String name, Date dob,
Address address,
2         String regNo, String
thesisTitle) {
3     super(name, dob, address,
regNo);
4     this.thesisTitle = thesisTitle;
5 }
```

super(...) always targets the direct superclass constructor — never “skip” a level.

Trace one object creation — calls go upward, completion comes downward

Example: new GradStudent("Talha", ..., "sp26", "research title")

Constructor calls

1 GradStudent(...) starts

↓

2 calls Student(...)

↓

3 calls Person(...)

Superclass state becomes valid before subclass construction finishes

Constructor bodies complete

4 Person body completes

↑

5 Student body completes

↑

6 GradStudent body completes

Mental model: a subclass constructor is responsible for its own new state, while its superclass constructor is responsible for superclass state.

Make the chain visible with print statements

This tiny experiment turns constructor chaining into observable runtime behavior.

TraceDemo.java

```java
class Person {
    Person() { System.out.println("Person"); }
}

class Student extends Person {
    Student() { System.out.println("Student"); }
}

class GradStudent extends Student {
    GradStudent() { System.out.println("GradStudent");
}
}
```

Predict before running

What is printed by:

new GradStudent();

Actual order

Person
Student
GradStudent

Why? The implicit constructor chain reaches the highest superclass first.

The classic compiler error: implicit super() cannot find a match

If you omit an explicit superclass-constructor call, Java still needs a valid direct-superclass constructor.

Broken version

```
1 class Person {
2     Person(String name) {
3         this.name = name;
4     }
5 }
6
7 class Student extends Person {
8     Student(String regNo) {
9         this.regNo = regNo;
10    }
11 }
```

What the compiler needs

Student must arrange for a Person constructor to run. Person() does not exist, so an implicit no-argument superclass call cannot succeed.

Fix

```
1 Student(String name, String regNo) {
2     super(name);
3     this.regNo = regNo;
4 }
```

Debugging question: “Which direct-superclass constructor is being selected?”

Overloaded superclass constructors give subclasses choices

super(arguments) selects the direct-superclass constructor using normal overload resolution.

Person.java

```
1 class Person {
2     Person(String name, Address address) { ...
}
3
4     Person(String name, Date dob, Address
address) { ... }
5 }
```

Student.java

```
 1 class Student extends Person {
 2     Student(String name, Address address, String regNo) {
 3         super(name, address);
 4         this.regNo = regNo;
 5     }
 6
 7     Student(String name, Date dob, Address address, String regNo) {
 8         super(name, dob, address);
 9         this.regNo = regNo;
10     }
11 }
```

Design idea

A subclass constructor can expose different construction paths while delegating shared superclass initialization to Person.

Avoid duplication

Do not copy Person’s field-initialization logic into Student. Let Person maintain its own invariants.

Constructor syntax note for modern Java

The conceptual rule is stable even though recent Java versions relaxed where super(...) may appear.

Traditional rule students often learn

JDK 21 and earlier: an explicit this(...) or super(...) invocation had to appear first in the constructor body. This is also the rule shown in most traditional textbooks/examples.

Java 25+ language update

Java 25+: limited statements may appear before an explicit super(...) / this(...) invocation, but the object under construction is restricted during this early-construction phase.

For this course and our live code: keep super(...) at the top unless there is a deliberate reason to teach the newer feature. The key invariant is that superclass construction is established before normal subclass initialization proceeds.

Oracle Java SE 25: Flexible Constructor Bodies (JEP 513) • Practical teaching choice: clarity first.

Private superclass state still exists — but the subclass should not manage it directly

Encapsulation continues across an inheritance relationship.

Person.java

```java
class Person {
    private String name;

    Person(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

Student.java

```java
class Student extends Person {
    private String regNo;

    void printStudent() {
        // System.out.println(name); // ERROR
        System.out.println(getName()); // OK
        System.out.println(regNo);
    }
}
```

The Student object has Person state; Student source code simply cannot bypass Person’s private boundary.

A strong inheritance pattern: superclass owns its invariants

A subclass reuses behavior rather than reaching into superclass representation.

Superclass responsibility

Person validates and stores name, date of birth, and address.

If the representation changes later, Person remains the authority over its own state.

Subclass responsibility

Student adds registration-specific state/behavior.

It uses the superclass API rather than duplicating or bypassing Person's rules.

Design benefit

Lower coupling between subclass code and the exact internal fields of the superclass.

This reduces ripple effects when Person changes.

PART III

Inheritance is also a design claim

extends does more than copy code. It claims that the subclass is a meaningful subtype of the superclass.

Predict → Trace → Compile → Explain

Two different ideas often get confused

Inheritance can reuse implementation, but its deeper meaning is a subtype relationship.

Goal A — implementation reuse

“I do not want to duplicate fields or methods.”

This can be useful, but by itself it is not enough reason to choose inheritance. Composition/delegation may reuse behavior without claiming IS-A.

Goal B — subtype meaning

“Every Student is a valid kind of Person.”

This is the semantic claim made by extends. The subclass should preserve the expectations attached to the superclass concept.

Best inheritance designs usually satisfy both: meaningful specialization + useful reuse.

Choose IS-A or HAS-A deliberately

A quick design test before touching the extends keyword.

Strong IS-A candidates

- GradStudent IS-A Student
- Student IS-A Person
- SavingsAccount IS-A BankAccount

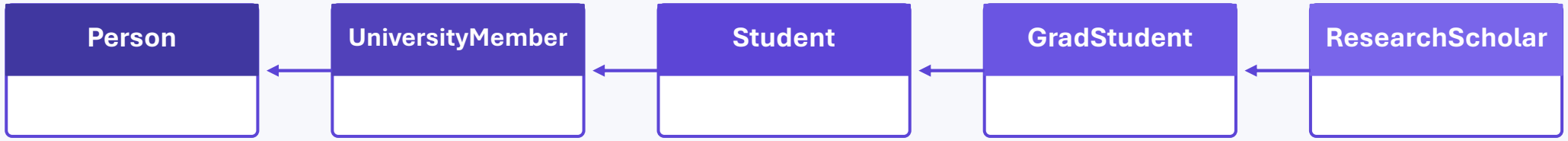
Likely HAS-A / USES-A instead

- Person HAS-A Address
- Car HAS-A Engine
- Course USES-A Room

Question to ask: “Would every object of the subclass still make conceptual sense wherever the superclass idea is expected?”

Why deep inheritance hierarchies become expensive

Each extra level adds inherited assumptions that developers must remember.



Reasoning cost

To understand ResearchScholar, you may need to inspect behavior and assumptions introduced by four ancestors.

Change cost

A change high in the hierarchy can affect many subclasses — sometimes in ways the superclass author did not anticipate.

Design warning

Do not add a level merely to “organize” code. Each level should represent a stable, useful abstraction.

Fragile base class — an intuition for beginners

Subclass code can quietly depend on superclass details, making later changes risky.

Version 1

```
1 class Person {
2     public void printSummary() {
3
System.out.println(getName());
4     }
5 }
```

Subclass

```
1 class Student extends Person {
2     // assumes
Person.printSummary() behavior
3 }
```


Later Person changes

Suppose Person.printSummary() changes formatting, calls new helper behavior, or strengthens an assumption.

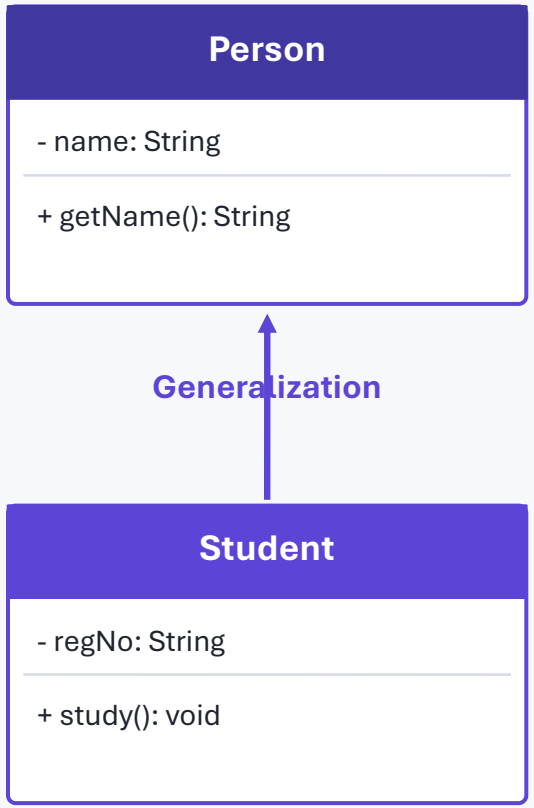
Student inherits the change automatically — even if Student was written against the old behavior.

The tighter the subclass depends on internals, the more fragile the hierarchy becomes.

Design response: keep superclass contracts clear and subclass coupling low.

UML generalization maps directly to Java extends

The hollow-triangle arrow points toward the more general class.



Java mapping

```java
public class Student extends Person {
    private String regNo;

    public void study() { ... }
}
```

Read UML first: Student specializes Person. Then map that relationship to extends.

Do not reverse the arrow: the triangle is on the superclass side.

Trace what each level introduces

A useful habit when reading any inheritance diagram.

Member / concept	Person	Student	GradStudent
name, dob, address	introduces	available through Person portion	available through Person portion
regNo	—	introduces	available through Student portion
thesisTitle	—	—	introduces
Person constructor	declares	invoked by Student constructor	reached through Student constructor
Student constructor	—	declares	invoked by GradStudent constructor

Use this tracing style before coding a new hierarchy: “What is common? What is newly introduced? Who owns initialization?”

PART IV

Debug, predict, and live-code

The fastest way to deepen inheritance understanding is to break the hierarchy deliberately and explain the compiler’s complaint.

Predict → Trace → Compile → Explain

Debugging challenge 1 — private field access

Predict the error before compiling. Then fix the design, not just the syntax.

Problem

```
1 class Person {
2     private String name;
3
4     public String getName() {
5         return name;
6     }
7 }
8
9 class Student extends Person {
10    void show() {
11        System.out.println(name);
12    }
13 }
```

Diagnosis

name belongs to Person’s private implementation. Student cannot access it directly.

Better fix

```
1 void show() {
2     System.out.println(getName());
3 }
```

Do not make every superclass field public just to “solve” inheritance errors.

Debugging challenge 2 — the missing constructor path

Constructor-chain errors are usually solved by tracing only the direct-superclass step.

Will this compile?

```
1 class Person {
2     Person(String name) { ... }
3 }
4
5 class Student extends Person {
6     Student() {
7         // no explicit super(...)
8     }
9 }
```

Step 1 — identify direct superclass

Student directly extends Person.

Step 2 — identify selected constructor

No explicit call means the compiler needs a valid no-argument Person construction path.

Step 3 — fix

Add a suitable Person() constructor or explicitly call super(name) from Student.

Design challenge — which hierarchy should we reject?

Inheritance errors can be semantic even when the code compiles perfectly.

A

class GradStudent extends Student

Meaning: every graduate student is a student.

B

class Student extends Address

Meaning: every student is an address.

C

class Person extends Object

Meaning: Person participates in Java's root class hierarchy.

B compiles only if Address is extendable — but the model is still wrong. Type correctness is not design correctness.

Recommended live-coding sequence using your existing files

Keep the same Person / Student / GradStudent example so students deepen one mental model instead of learning a new scenario.

1	**Add print statements to all three constructors**	Observe call/completion order
2	**Remove explicit super(...) from Student**	Trigger constructor-chain error
3	**Add a no-arg Person() temporarily**	See why implicit super() now works
4	**Make Person fields private + add getName()**	Show encapsulation across inheritance
5	**Add a second Student constructor**	Use different Person overloads
6	**Draw UML from final code**	Verify arrows and introduced members

Teaching rhythm: ask for prediction before every compile/run step.

In-class reasoning exercise — no IDE for the first 3 minutes

Students should answer on paper, then verify by compiling.

Question code

```
1  class A {
2      A(int x) { System.out.println("A " + x); }
3  }
4
5  class B extends A {
6      B() {
7          super(5);
8          System.out.println("B");
9      }
10 }
11
12 class C extends B {
13     C() { System.out.println("C"); }
14 }
```

Q1

What prints for new C()? Explain every line in terms of constructor chaining.

Q2

Would C compile if B had only B(String s)? Why?

Answer to Q1: A 5 → B → C

The inheritance mental model to keep

Five statements should now be automatic.

1. **extends makes a subtype claim, not merely a reuse request.**
2. A subclass object contains state contributed by its superclass chain.
3. Constructors are not inherited; superclass construction is chained.
4. Private superclass fields exist in the object but stay behind the superclass boundary.
5. Shallow, meaningful hierarchies are easier to reason about and safer to evolve.

Next lecture: subclasses refine behavior — overriding, @Override, protected, final, and super.method(...).

LECTURE 13

Exit ticket — 4 questions

Use these to check whether the mental model is ready for method overriding.

Q1

Why does creating GradStudent cause Person construction to run?

Q2

What exact problem occurs if Person has Person(String name) but Student() does not call super(name)?

Q3

Can Student directly access Person.privateName? If not, does that state still exist in a Student object?

Q4

Give one example where composition is better than inheritance, and explain the relationship in one sentence.

Next: method overriding, safe specialization, String immutability, and wrappers.

Part V — Method overriding

Specialize inherited behavior without duplicating an entire class

OVERRIDING

Part V — Method overriding

Specialize inherited behavior without duplicating an entire class

From inheritance to overriding

Inheritance reuses behavior; overriding replaces an inherited instance-method implementation.

Base class: Person

```
class Person {
  String role() {
    return "Person";
  }
}
```

Subclass: Student

```
class Student extends Person {
  @Override
  String role() {
    return "Student";
  }
}
```

Student inherits the method contract, but supplies its own implementation.

What makes this an override?

Four questions to ask whenever you see a method with the same name in two classes.

Checklist

1. Does the superclass have an inheritable instance method?
2. Does the subclass use the same name and parameter types?
3. Is the return type compatible?
4. Is access at least as permissive?

Example

```
class Person {
  public String info() {
    return "Person";
  }
}
class Student extends Person {
  @Override
  public String info() {
    return "Student";
  }
}
```

Overloading ≠ overriding

Students often confuse signature changes with subclass specialization.

Overloading

Same method name; different parameter lists.

May occur in one class or across classes.

Selection depends on compile-time argument types.

```
void print() { }
void print(String label) { }
```

Overriding

Subclass redeclares inherited instance method with compatible signature.

Method body specializes behavior.

Runtime object determines implementation for virtual instance calls.

```
@Override
void print() { }
```

@Override catches mistakes

It asks the Java compiler to verify that a method really overrides a superclass method.

Common mistake

```
class Person {
  void display() { }
}
class Student extends Person {
  @Override
  void dispaly() { }
}
```

What happens?

Compilation error: method does not override a method from a supertype.

Without @Override, the misspelled method would instead be a NEW method.

Practice: fix dispaly → display.

super.method() versus super(...)

Both use super but solve different problems.

super(...) — construction

```
class Student extends Person {
  Student(String name) {
    super(name);
  }
}
```

Invokes direct superclass constructor;
not an ordinary method call.

super.method(...) — behavior

```
class Student extends Person {
  @Override
  void show() {
    super.show();
    System.out.println("ID");
  }
}
```

Invokes superclass implementation.

Extend rather than replace superclass behavior

A subclass may preserve the base implementation, then add useful specialized output.

Person

```
class Person {
  void show() {
    System.out.println("Name");
  }
}
```

Student

```
class Student extends Person {
  @Override
  void show() {
    super.show();
    System.out.println("Reg No");
  }
}
```

Question: what two lines print for new Student(...).show()?

Overriding rules: return type and access

Java rejects overrides that violate the superclass method contract.

Valid return type

```
class Person {
  Person copy() { return this; }
}
class Student extends Person {
  @Override
  Student copy() { return this; }
}
```

A covariant return type is allowed.

Invalid access narrowing

```
class Person {
  public void show() { }
}
class Student extends Person {
  // ERROR: weaker access
  protected void show() { }
}
```

Also: incompatible primitive return types are not allowed.

Other overriding constraints

Understand which operations may or may not be overridden.

Can be overridden

Inherited, overridable instance methods.

Protected or public access (as appropriate).

Package-private when accessible/inherited.

Checked exceptions cannot be widened in the override.

Cannot be overridden

final instance methods.

private methods (not inherited; same-named subclass method is distinct).

static methods (hidden, not overridden).

constructors (not methods and not inherited).

What does protected mean?

Access is more nuanced than “visible to all subclasses everywhere.”

Same package

protected members are accessible to code in the same package (subject to ordinary Java access rules).

Subclass can also inherit them.

Prefer private fields + public/protected methods where suitable.

Different package

A subclass can access an inherited protected instance member through this/super (or an appropriately typed subclass reference).

It cannot freely access it through any arbitrary superclass instance.

final — restrict extension intentionally

Use final to protect invariants or prevent unwanted specialization.

Final method

```
class Person {
  public final String getId() {
    return "ID";
  }
}

Student cannot override getId().
```

Final class

```
final class Address {
  // immutable value object candidate
}

A final class cannot be extended.

A final field cannot be reassigned once
assigned.
```

Part VI — Subtypes & dynamic dispatch

A first look at polymorphism using the hierarchy already studied

POLYMORPHISM

Part VI — Subtypes & dynamic dispatch

A first look at polymorphism using the hierarchy already studied

Superclass reference, subclass object

The reference type and the actual object type can differ.

Working example

```
Person p = new Student(...);
p.show();
```

The object is Student.

The variable p has type Person.

Accessible method names are checked using the declared reference type.

Runtime method selection

If show() is overridden, the Student version executes.

This is dynamic dispatch for overridable instance methods.

It does NOT mean every field or static method uses runtime dispatch.

Trace three calls: predict first

Assume each class overrides describe() using @Override.

Code

```
Person a = new Person(...);
Person b = new Student(...);
Person c = new GradStudent(...);

a.describe();
b.describe();
c.describe();
```

Predicted behavior

a → Person.describe()

b → Student.describe()

c → GradStudent.describe()

Runtime object type determines which overriding implementation runs.

Student-specific methods cannot be called directly through Person-typed b.

Fields are hidden, not overridden

Do not infer field selection from method overriding.

Example

```
class Person { String tag = "P"; }
class Student extends Person {
  String tag = "S";
}
Person p = new Student();
System.out.println(p.tag);
```

Outcome and rule

Output: P

Field access is resolved using the compile-time reference type.

Avoid redeclaring inherited field names; favor private state and methods.

The role of Object and toString()

Every ordinary Java class ultimately derives from java.lang.Object.

Without custom override

```
Student s = new Student(...);
System.out.println(s);
```

The inherited Object.toString() typically prints a class name plus hexadecimal hash-code representation.

With custom override

```
@Override
public String toString() {
  return "Student: " + regNo;
}
```

Now printing the object produces informative text.

Use @Override and the public access level.

equals() and == are not equivalent

A useful related concept for both object design and String.

Reference identity

```
Object a = new Object();
Object b = new Object();
System.out.println(a == b);
// false: separate objects
```

Logical equality

```
String x = new String("Java");
String y = new String("Java");
System.out.println(x.equals(y));
// true: equal text content
```

Designing equals() for custom classes requires a consistent contract with hashCode().

Inheritance versus composition: final design check

Do not use inheritance merely to share a few fields.

Good IS-A

GradStudent IS-A Student.

Student IS-A Person.

A GradStudent should support the general Student behavior expected by callers.

Good HAS-A

Person HAS-A Address.

Person HAS-A Date (birth date).

Address and Date are separate objects referenced through fields, not superclasses.

Fragile base class: overriding can change assumptions

An inherited call may dispatch to subclass behavior unexpectedly.

Conceptual risk

Base constructor or method calls an overridable method.

The subclass override runs before subclass initialization is complete.

This can expose partially initialized state.

Safer design

Avoid calling overridable instance methods from constructors.

Keep constructors focused on establishing valid state.

Favor small, stable hierarchies and explicit extension points.

Part VII — String is immutable

CDF reinforcement: identity, references, new strings and the string pool

STRING

Part VII — String is immutable

CDF reinforcement: identity, references, new strings and the string pool

What does immutable mean?

A String object’s character sequence cannot be changed after creation.

Example

```
String s = "Java";
s.concat(" OOP");
System.out.println(s);

// Java
```

Why?

concat produces a new String result (when changed).

Since the result was not assigned, s still references "Java".

Assign the result:

s = s.concat(" OOP");

References can change; String objects do not

Reassignment of a variable is different from mutation of an object.

Step by step

```
String a = "Hello";
String b = a;
a = a + " Java";

System.out.println(a);
System.out.println(b);
```

Reference reasoning

Initially a and b refer to the same immutable text.

After concatenation a refers to a different String.

b still refers to "Hello".

Expected: Hello Java / Hello.

String pool, literals and new String

Distinguish reference equality from content equality.

Code

```
String a = "OOP";
String b = "OOP";
String c = new String("OOP");

System.out.println(a == b);
System.out.println(a == c);
System.out.println(a.equals(c));
```

Results

true

false

true

String literals with the same content are interned.

new String(...) creates a distinct String instance.

Use equals() to compare text content.

String API: results vs mutations

These operations return values; they do not edit an existing String object.

Code

```
String s = "  Java OOP  ";
String t = s.trim();
String u = t.toUpperCase();

System.out.println(s);
System.out.println(t);
System.out.println(u);
```

Thinking task

How many variables hold String references?

Which one still has spaces?

Does toUpperCase() mutate t?

Related methods: substring(), replace(), charAt(), length().

Why String immutability is useful

Tie immutability back to object state and program design.

Advantages

A String may be safely shared without callers changing its content.

Stable text supports reliable use as a map key.

The model is easier to reason about when values cannot mutate.

Trade-offs

Repeated concatenation in loops can create many intermediate strings.

For repeated in-place text-building operations, use StringBuilder (a mutable text builder).

This is not a subclass of String.

Is String a final class?

Connect the CDF String topic to the inheritance rules.

Java declaration concept

java.lang.String is final.

You cannot write:

class MyString extends String { }

This would be a compile-time error.

Design link

A final class disallows subclassing.

String immutability also depends on its internal implementation and lack of mutating operations — final alone does not guarantee immutability.

Part VIII — Wrapper classes

CDF reinforcement: primitives, objects, boxing, unboxing, and common pitfalls

WRAPPERS

Part VIII — Wrapper classes

CDF reinforcement: primitives, objects, boxing, unboxing, and common pitfalls

Primitive types versus wrapper objects

A wrapper class lets us represent a primitive value as an object.

Eight pairs

byte → Byte

short → Short

int → Integer

long → Long

float → Float

double → Double

char → Character

boolean → Boolean

Why learn them?

They provide object-based APIs around primitive values.

Wrappers are useful where an object reference is required, including generic collections.

The wrapper objects are immutable.

Autoboxing and unboxing

Java often inserts conversions automatically.

Code

```
int n = 15;
Integer obj = n;        // boxing
int m = obj;            // unboxing

System.out.println(m);
```

Equivalent idea

```
Integer obj = Integer.valueOf(n);
int m = obj.intValue();
```

Be careful: automatic conversion does not make primitive and wrapper types identical.

Null unboxing can fail at runtime

Compilation may succeed even though unboxing will throw an exception.

Code

```
Integer count = null;
int n = count;

// NullPointerException
// during unboxing
```

Reasoning

count holds no Integer object.

The conversion requires extracting a primitive value from an object.

Check for null or avoid nullable wrappers where appropriate.

Integer equality: equals() vs ==

This comparison is a classic OOP quiz trap.

Code

```
Integer a = 200;
Integer b = 200;
System.out.println(a == b);
System.out.println(a.equals(b));
```

Reliable interpretation

equals(): true (same numeric value)

== compares references when both operands are Integer objects; reference identity is not guaranteed.

Do not rely on wrapper caching to compare numeric values.

Parsing, conversion, and numeric utilities

Use documented wrapper methods rather than inventing conversion logic.

Common expressions

```
int n = Integer.parseInt("42");
Integer x = Integer.valueOf("42");
double d = Double.parseDouble("3.5");
String s = Integer.toString(42);
```

Failure behavior

Integer.parseInt("hello") throws NumberFormatException.

Integer.valueOf returns an Integer object.

Integer.parseInt returns a primitive int.

Input validation remains important.

Wrappers are immutable; variables can be reassigned

As with String, distinguish value immutability from reference reassignment.

Example

```
Integer a = 7;
Integer b = a;
a = a + 1;

System.out.println(a); // 8
System.out.println(b); // 7
```

Meaning

The original Integer object holding 7 was not changed.

The expression a + 1 involves unboxing, arithmetic and boxing.

a is reassigned to an Integer representing 8.

Wrapper inheritance: useful but limited

Connect wrappers to the broader class hierarchy without introducing interfaces.

Hierarchy facts

Number is an abstract superclass of Integer, Double, Float, Long, Short and Byte.

Character and Boolean do not extend Number.

All wrapper classes ultimately extend Object.

Design caution

The wrapper classes are final.

For example, class MyInt extends Integer is not allowed.

Do not treat wrappers as mutable containers for primitives.

Part IX — Integration & practice

Apply inheritance rules, String immutability and wrappers in one session

PRACTICE

Part IX — Integration & practice

Apply inheritance rules, String immutability and wrappers in one session

Predict the output — overriding + reference type

Decide which method runs and which expressions are legal.

Code

```
Person p = new GradStudent(...);
System.out.println(p.toString());
// p.thesisTitle = "X";

Assume toString() is overridden by
GradStudent.
```

Discuss

A GradStudent override runs for a virtual toString() call.

The Person reference does not expose a GradStudent-specific field directly.

Explain static accessibility versus runtime method dispatch.

Predict the output — Strings and wrappers

Write outputs before compiling the code.

Snippet A

```
String a = "CS";
String b = a;
a += "C241";
System.out.println(b);
System.out.println(a);
```

Snippet B

```
Integer x = 5;
Integer y = x;
x++;
System.out.println(x);
System.out.println(y);
```

Expected: A → CS then CSC241. B → 6 then 5.

Debug these three errors

Have students identify the relevant Java rule, not merely replace the syntax.

Problems

A. @Override int show(String s) {}
where superclass has void show()

B. class X extends String {}

C. Integer n = null; int x = n;

What to explain

A. No matching inherited method signature; @Override detects mistake.

B. String is final, so it cannot be extended.

C. Unboxing null throws NullPointerException.

Which failures occur at compile time? Which at runtime?

Suggested live-coding continuation

Keep the exact user-provided source hierarchy; incrementally compile after each step.

Inheritance upgrades

1. Add show() to Person.
2. Override it in Student.
3. Override again in GradStudent.
4. Add @Override and super.show().
5. Add a final superclass method.

CDF reinforcement

6. Demonstrate String reference reassignment.
7. Compare literals with == and equals().
8. Convert String registration year using Integer.parseInt().
9. Trigger and fix null-unboxing issue.

Final review — ten exam-ready statements

Use this as an exit ticket or short formative assessment.

1. Inheritance expresses IS-A, not HAS-A.
2. Constructors are chained, not inherited.
3. super(...) selects a direct superclass constructor.
4. @Override detects incorrect override attempts.
5. super.method() invokes the superclass implementation.
6. A subclass cannot reduce inherited method visibility when overriding.
7. final methods cannot be overridden; final classes cannot be extended.
8. Overridden instance methods use runtime dispatch.
9. String is immutable, and equals() compares its content.
10. Wrapper classes support boxing/unboxing; null unboxing fails.

References and lecture boundaries

Source-aligned coverage, with the main examples retained from your existing code.

Course: CSC241 Object-Oriented Programming (CUI Lahore), Fall 2026.

Teaching model: Person → Student → GradStudent; Person HAS-A Address and Date.

Official course coverage: inheritance, overriding, class design, immutable String, wrapper classes.

Reference reading: Y. Daniel Liang, Introduction to Java Programming and Data Structures, Ch. 9–11 (edition-dependent chapter numbering).

Not covered in this presentation: interfaces, interface inheritance, functional interfaces, generics implementation.

Detailed casts and substitution-contract analysis can be developed in subsequent lectures.